

P-RFO Notes

Partitioned Rational-Function Optimization for Saddle Search

Notes for ChemDM

A conceptual reference for partitioned rational-function optimization as implemented in `src/chemdm/prfo.py`. Captures the *why*, not just the *what* — in particular why BFGS fails for saddle search, where the “shift” in P-RFO comes from, and why this is a saddle *finder* and not a reaction *path* method.

1 The problem

Find a first-order saddle point of a potential energy surface $E(x)$ using *only forces* $-\nabla E$, no analytic Hessian. The first-order saddle has

$$\nabla E = 0, \quad H = \nabla^2 E \text{ has exactly one negative eigenvalue.}$$

Two challenges: (a) the gradient alone is zero at every minimum *and* every saddle, so gradient-following cannot distinguish them, and (b) the Hessian is expensive — the very thing we want to avoid computing.

2 Why BFGS specifically fails

The BFGS quasi-Newton update

$$H_{k+1} = H_k + \frac{yy^T}{y^T s} - \frac{H_k s s^T H_k}{s^T H_k s}, \quad s = x_{k+1} - x_k, \quad y = g_{k+1} - g_k,$$

provably preserves positive definiteness when the curvature condition $y^T s > 0$ holds. Line search enforces this condition for minimization.

The consequence for saddle search is fatal: if $H_0 = I$ (PD), every subsequent H_k is PD by induction. The lowest eigenvalue can never cross zero. The Newton step $-H_k^{-1}g$ always points downhill. **You converge to a minimum, no matter how close you start to a saddle.**

This is not a corner case — it is a fundamental structural mismatch. BFGS was designed for minimization and bakes that intent into its update rule.

The fix is to use a quasi-Newton update that *does not preserve PD*:

Update	PD-preserving	Used for
BFGS	yes	minimization
SR1 (symmetric rank-1)	no	saddle search (unstable if $r^T s$ small)
PSB (Powell-symmetric-Broyden)	no	saddle search (stable, slower)
Bofill (SR1/PSB convex mix)	no	saddle search (best of both)

`prfo.py` uses Bofill with weight $\varphi = (r^T s)^2 / (\|r\|^2 \|s\|^2)$ — the cosine squared between the residual $r = y - H_k s$ and s . SR1-heavy when those vectors align, PSB-heavy when they do not.

3 Newton with an indefinite Hessian — almost works

Suppose we *had* the true indefinite Hessian at a saddle. Decompose the Newton step $\Delta x = -H^{-1}g$ in the eigenbasis:

$$\Delta x_i = -\frac{g_i}{\lambda_i}.$$

For positive λ_i , Δx_i opposes g_i — descent. For *negative* λ_i , Δx_i aligns with g_i — **ascent**. So Newton with the true Hessian automatically climbs the unstable mode and descends the others. This is eigenvector-following before we even introduce RFO.

But two problems remain:

1. **Step blows up near singular Hessians.** Any $\lambda_i \approx 0$ makes $|\Delta x_i| = |g_i/\lambda_i|$ enormous. We need regularization.
2. **Wrong-mode following.** Early in the search, the “naturally” most negative eigenvalue may not be the one we actually want to ascend (multiple soft modes, eigenvalues crossing zero during the search). We need explicit *control* over which mode to follow.

RFO solves both at once.

4 The RFO step as shift-invert

Replace the quadratic energy model with a **rational function**:

$$\Delta E(\Delta x) \approx \frac{g^T \Delta x + \frac{1}{2} \Delta x^T H \Delta x}{1 + \Delta x^T \Delta x}.$$

Stationary points of this rational function are solutions of the **augmented eigenvalue problem**:

$$\begin{pmatrix} H & g \\ g^T & 0 \end{pmatrix} \begin{pmatrix} \Delta x \\ 1 \end{pmatrix} = \mu \begin{pmatrix} \Delta x \\ 1 \end{pmatrix}. \quad (1)$$

Expanding the first block row gives $H\Delta x + g = \mu\Delta x$, so

$$\boxed{\Delta x = -(H - \mu I)^{-1}g.} \quad (2)$$

That is shift-invert. The Hessian gets shifted by μ , inverted, applied to $-g$. The same operator appears in:

- **Levenberg-Marquardt** for nonlinear least squares (shift = damping α);
- **Trust-region Newton (Moré-Sorensen)** — shift chosen to satisfy $\|\Delta x\| = \text{trust_radius}$;
- **Tikhonov regularization** for ill-posed inverse problems;
- **Inverse power iteration / shift-invert Lanczos** for eigenvalue problems.

What makes RFO distinctive among shift-invert methods is the **rule for picking μ** : it is an eigenvalue of the augmented $(N+1) \times (N+1)$ matrix in equation (1). Equivalently, substituting Δx into the second row of (1) gives the **secular equation**:

$$\sum_i \frac{f_i^2}{\mu - \lambda_i} = \mu, \quad (3)$$

where λ_i are eigenvalues of H and $f_i = (\text{eigvec}_i)^T g$.

This scalar equation has $N + 1$ roots, interleaved with the λ_i 's plus one below the lowest and one above the highest eigenvalue. **Each root is a valid shift; each gives a different step type:**

- **Lowest root** $\mu < \lambda_{\min}$: all $(\lambda_i - \mu) > 0$, so the step descends in every mode. Minimization RFO.
- **Largest root** $\mu > \lambda_{\max}$: all $(\lambda_i - \mu) < 0$, so the step ascends in every mode. Not usually what you want.
- **Intermediate roots**: mixed — ascends in some modes, descends in others.

The choice of root encodes the method's intent.

The rational denominator $1 + \|\Delta x\|^2$ is what gives RFO its **automatic step bounding**. As $\|g\|$ grows or H becomes singular, μ grows in magnitude so $(\lambda_i - \mu)$ stays bounded away from zero. The step Δx cannot blow up by construction — no trust-region constraint required. (We still impose one as a second safety net in `prfo.py`, used adaptively via the $\rho = \Delta E_{\text{actual}}/\Delta E_{\text{predicted}}$ ratio.)

5 The partitioning (the “P” in P-RFO)

For transition-state search we want one mode to ascend and the rest to descend. **Use two different shifts.**

Followed (ascent) mode k . Solve the 2×2 RFO subproblem in mode k alone:

$$\begin{pmatrix} \lambda_k & f_k \\ f_k & 0 \end{pmatrix} \begin{pmatrix} v \\ 1 \end{pmatrix} = \mu_+ \begin{pmatrix} v \\ 1 \end{pmatrix}.$$

Take the *larger* root,

$$\mu_+ = \frac{1}{2} \left(\lambda_k + \sqrt{\lambda_k^2 + 4f_k^2} \right) > \lambda_k.$$

Then $\Delta q_k = -f_k/(\lambda_k - \mu_+)$ ascends along mode k .

Descent subspace (the other $N - 1$ modes). Solve the $M \times M$ RFO subproblem on the remaining eigenvalues; take the *lowest* root $\mu_- < \min_{j \neq k} \lambda_j$. Then $\Delta q_j = -f_j/(\lambda_j - \mu_-)$ descends in each remaining mode.

Reassemble Δq in the eigenbasis and rotate back to get Δx . **One ascent mode, all-others descent.** That is the P in P-RFO.

Mode-following heuristic. The choice of which mode k to follow: on iteration 1, pick the lowest eigenvalue (most negative if you have one, otherwise softest). Subsequent iterations: pick the eigenvector with maximum overlap with the previously-followed eigenvector. Failure mode: when the followed eigenvalue crosses zero or near-degenerate eigenvalues swap, overlap can pick the wrong mode and the optimizer briefly walks the wrong way. Our HCN demo's iterations 19–31 show this in action — the optimizer recovers, but it is the main source of fragility in the current `prfo.py`.

6 Coordinate systems are orthogonal to RFO

RFO is just linear algebra on (H, g) . It works in any coordinate system. What changes between coordinate systems is the *plumbing around* RFO, not RFO itself.

Coords	Dim	Zero modes	Back-transform
Cartesian + trans/rot projection	$3N$	6 (project out)	none
Non-redundant internals	$3N - 6$	0	iterative Wilson B
Redundant internals (Pulay)	$M > 3N - 6$	$M - (3N - 6)$	Wilson B + pseudoinverse

The **three-axis design space** for any TS optimizer is:

Step generator	{Newton, trust-region Newton, RFO/P-RFO, dimer rotation}
Hessian model	{analytic, Bofill, SR1, PSB, BFGS (minimization only)}
Coordinate system	{Cartesian + projection, internal, redundant internal}

Most published “methods” are just specific points in this cube. Our `prfo.py` is {P-RFO, Bofill, Cartesian + projection}. Gaussian’s TS optimizer is {P-RFO, Bofill, redundant internals}. Sella is roughly {trust-region Newton, exact Hessian via finite differences, Cartesian + projection}. Dimer codes are {dimer rotation step, no stored Hessian, Cartesian}.

These choices are independent — you can swap any axis without changing the others.

7 P-RFO finds *points*, not *paths*

The optimizer’s iterates trace a discrete trajectory through configuration space, but **that trajectory has no chemical meaning**. It depends on initial guess, Hessian model, trust radius, mode-following heuristic — none of which are properties of the molecule. Two runs from different starts can take very different optimizer paths to the same saddle.

The “eigenvector-following” name refers to following an *eigenvector at each iteration*, not to following a *curve through configuration space*. These are different objects.

The reaction path is a separate computation, defined intrinsically by the PES:

IRC (Intrinsic Reaction Coordinate, Fukui) From the converged saddle, integrate $dx/dt = -g(x)$ in mass-weighted Cartesians, $\pm$ along the unstable mode. One direction reaches the reactant minimum, the other the product minimum. This curve is geometry-defined; different optimizers that find the same TS yield the same IRC.

This cleanly separates concerns:

Step	Method	Object computed	Cost
1	P-RFO + Bofill	Saddle point (single geometry)	~ 20 – 100 force calls
2	IRC (steepest descent $\pm$)	Reaction path (a curve)	~ 50 – 200 force calls

Both are needed for a complete reaction characterization. P-RFO alone gives you a saddle that might connect anywhere; IRC without a converged TS has no starting point.

8 A note on gradient extremals (and why we do not build them)

Gradient extremal curves (Hoffman, Nord, Ruedenberg) are an alternative class of methods that *do* try to trace a curve directly from the reactant basin. A gradient extremal is the locus of points where ∇E is parallel to a specific Hessian eigenvector.

There is a connection: the discrete trajectory of P-RFO with mode-following is approximately a gradient extremal of the followed mode. So in a loose sense, P-RFO is discretely walking *along* a gradient extremal toward its endpoint at the saddle.

Gradient extremal *following* methods (continuous ODE integration of the curve, adaptive parametrization, branch tracking) make the *curve itself* the object of interest — you walk from the reactant minimum along the curve, through any saddles you encounter, and into the product basin. This is elegant and chemically motivated, but practically harder than the “find TS then IRC” approach for several reasons:

- Gradient extremals branch and recombine — choosing the right branch at every bifurcation requires bookkeeping that IRC does not.
- They can leave the chemically relevant region of the PES, especially in high-dimensional systems.
- They do not necessarily pass through every saddle, and a saddle they do not pass through is invisible to the method.
- Numerically: ODE integration with branch tracking is heavier than “find a stationary point, then steepest-descent.”

Our practical choice is the simpler decomposition:

relaxed reactant $\rightarrow$ `estimate_lowest_mode` (Lindh + Lanczos) $\rightarrow$ perturb + P-RFO $\rightarrow$ IRC.

The IRC step is cheap enough (gradient descent, ~ 150 lines of code) that we do not gain anything from trying to construct the reaction path directly during the saddle search. The gradient-extremal connection is a real mathematical fact, but it does not pay off in code.

9 Summary

P-RFO is **shifted Newton with self-determined shifts**, where the shifts come from an augmented eigenvalue problem and partition configuration space into one ascent mode and a descent subspace. The Hessian is approximated by Bofill (an indefinite-capable quasi-Newton update — BFGS does not work here). The step is implicitly bounded by the rational-function model’s denominator; we add an explicit trust radius as a second safety net with adaptive shrink/grow.

This finds a single saddle point per run. To trace the reaction path that connects the saddle to its endpoints, run IRC from the converged TS — a separate, cheap, well-defined computation.

For posterity: the three-axis design space (step generator $\times$ Hessian model $\times$ coordinate system) is what makes the TS-optimization literature navigable. Once you see it that way, “method X vs. method Y” is mostly a question of which point in the cube each method occupies.

10 Postscript: minimization vs. TS is one root choice

The cleanest possible statement of what RFO does: **minimization and transition-state search differ only by which root of the same secular equation you pick.** Everything else — Hessian update, Bofill, trust radius, coordinate system — is shared infrastructure.

Recall the secular equation (3):

$$\sum_i \frac{f_i^2}{\mu - \lambda_i} = \mu.$$

By the interlacing property of its $N + 1$ roots, they sit between the Hessian eigenvalues plus one outside on each end:

$$\mu_0 < \lambda_1 < \mu_1 < \lambda_2 < \mu_2 < \lambda_3 < \cdots < \lambda_N < \mu_N.$$

Each choice of root gives a step of the form $\Delta x = -(H - \mu I)^{-1}g$. What changes is the *sign pattern* of $(\lambda_i - \mu)$ across modes:

Root chosen	Sign pattern of $(\lambda_i - \mu)$	Step type
$\mu_0 < \lambda_1$	all positive	minimization
$\mu_1 \in (\lambda_1, \lambda_2)$	one negative, rest positive	TS, follow mode 1
$\mu_2 \in (\lambda_2, \lambda_3)$	two negative, rest positive	2nd-order saddle search

So, to answer the practical question: **starting from a local minimum, pick $\mu = \mu_1$ instead of μ_0 and the optimizer automatically pursues a transition-state search along the softest mode.** No fake-negative-eigenvalue initialization required, no special-case code, no different update rule. Same Hessian, same gradient, same secular equation — different root.

Two practical caveats:

1. At a true local minimum, $\|g\| = 0$ exactly, so all $f_i = 0$ and the secular equation degenerates: every $\mu_i = \lambda_i$, every step is zero. You must perturb slightly off the minimum to give the optimizer something to climb (and to break mode degeneracy in symmetric systems). This is what the demo's `x_start = x_min + kick * u` step does.
2. The two-shift formulation in Section 5 (μ_+ from a 2×2 subproblem on the followed mode, μ_- from an $M \times M$ subproblem on the rest) is mathematically equivalent to picking μ_1 here. The partitioned form is what production codes implement for numerical reasons (smaller eigenproblems, cleaner mode-following control), but the underlying mathematical content is “pick the secular root that sits above the eigenvalue you want to ascend.”

This is the unifying picture: P-RFO and ordinary RFO minimization are the *same algorithm* running on the *same data structure*, choosing a different root of the same scalar equation. When you see it that way, the whole machinery of TS search collapses into a one-line modification of an ordinary minimizer.

References

- Schlegel, *WIREs Comput. Mol. Sci.* **1**, 790 (2011). Best tutorial review.
- Banerjee, Adams, Simons, Shepard, *J. Phys. Chem.* **89**, 52 (1985). The original P-RFO paper.

- Baker**, *J. Comput. Chem.* **7**, 385 (1986). The practical recipe (mode following by max overlap) most QC codes follow.
- Bofill**, *J. Comput. Chem.* **15**, 1 (1994). The Bofill update.
- Besalú & Bofill**, *Theor. Chem. Acc.* **100**, 265 (1998). Proper trust-radius treatment for RFO.
- Wales**, *Energy Landscapes* (Cambridge, 2003), ch. 6–7. Wide-angle view connecting RFO to dimer, GAD, and gradient extremals.
- Hoffman, Nord, Ruedenberg**, *Theor. Chim. Acta* **69**, 265 (1986). Gradient extremal curves (for reference; we do not implement these).